Algoritmos e Estruturas de Dados II: questões de provas

As questões repetidas entre as provas aparecem uma vez só. Depois de cada questão há espaço para resolver. O gabarito comentado está no final.

Questão 1 *(lista duplamente encadeada circular, C)*

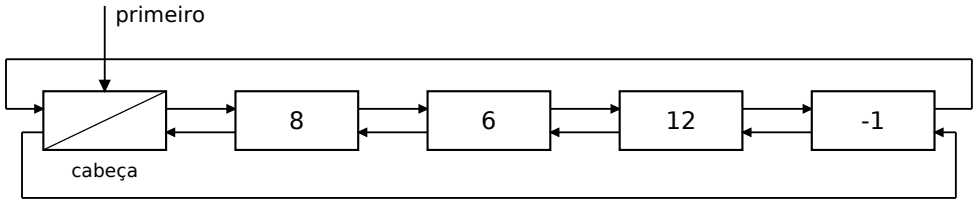
Implemente a função **void inserirFim(int x)** em linguagem C para uma Lista Duplamente Encadeada Circular com Célula Cabeça. A lista é representada apenas pelo ponteiro **primeiro**, que aponta para a célula cabeça (sentinela); a célula cabeça não guarda dados válidos. A lista é circular, ou seja, o ponteiro **ant** da célula cabeça aponta para a última célula da lista, e o ponteiro **prox** da última célula aponta para a célula cabeça. Apresente, em função de Θ, a complexidade do seu método.

```
typedef struct CelulaDupla {
    int elemento;               // Elemento inserido na celula.
    struct CelulaDupla* prox;   // Aponta a celula prox.
    struct CelulaDupla* ant;    // Aponta a celula anterior.
} CelulaDupla;

CelulaDupla* novaCelulaDupla(int elemento) {
    CelulaDupla* nova = (CelulaDupla*) malloc(sizeof(CelulaDupla));
    nova->elemento = elemento;
    nova->ant = nova->prox = NULL;
    return nova;
}

// LISTA PROPRIAMENTE DITA
CelulaDupla* primeiro;

void inserirFim(int x) {
    // implementar aqui
}
```



Resolução:

Questão 2 *(lista simples, C)*

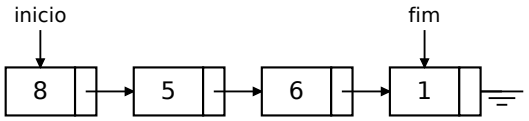
Dada uma lista simplesmente encadeada, implemente a função **void meiose()**. Esta função deve percorrer toda a lista, dividindo cada célula em duas, com cada nova célula contendo a metade do valor da célula original. Abaixo estão a estrutura do código e uma ilustração da lista antes e depois da operação. Apresente a complexidade do código em função de Θ.

```
typedef struct Celula {
    int elemento;
    struct Celula* prox;
} Celula;

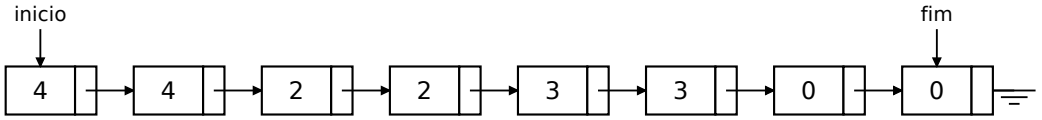
Celula* inicio;
Celula* fim;

void meiose() {
    // implemente a funcao aqui
}
```

Antes:



Depois:



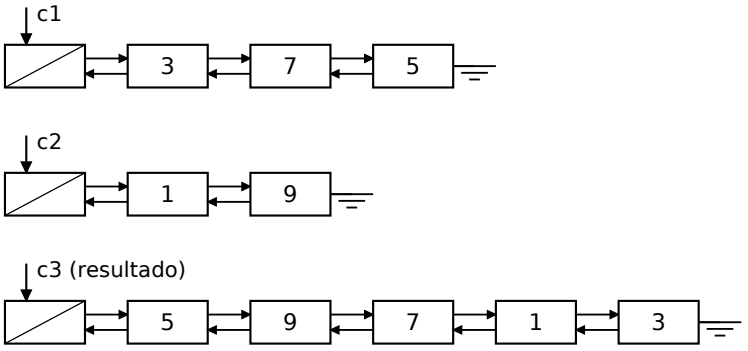
Resolução:

Questão 3 *(lista duplamente encadeada, Java)*

Implemente um método que receba o nó cabeça de duas listas duplamente encadeadas e retorne o nó cabeça de uma nova lista, resultante da **intercalação reversa** dos elementos das duas listas, ou seja, partindo do último para o primeiro elemento de cada lista. A nova lista deve ser construída intercalando um elemento do final de uma lista com um elemento do final da outra, alternadamente. Considere o código a seguir como base e observe o exemplo.

```
class CelulaDupla {
    int elemento;
    CelulaDupla prox, ant;
}

CelulaDupla intercalarReverso(CelulaDupla c1, CelulaDupla c2) {
    // IMPLEMENTAR
}
```



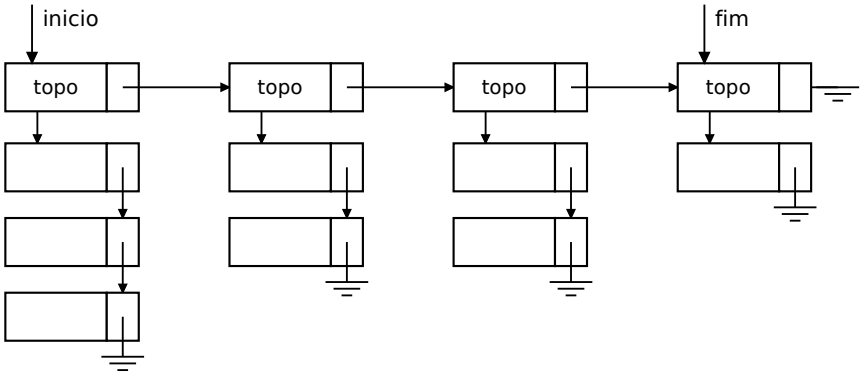
A primeira célula de cada lista (riscada) é a célula cabeça.

Resolução:

Questão 4 *(lista de pilhas, Java)*

Considere uma estrutura Lista de Pilhas (ilustrada e codificada abaixo). Crie o método **CelulaLista maiorPilha()** na classe Lista, que retorna a célula da Lista que aponta para a pilha com o maior número de elementos. Caso existam pilhas do mesmo tamanho, retornar a primeira que aparece.

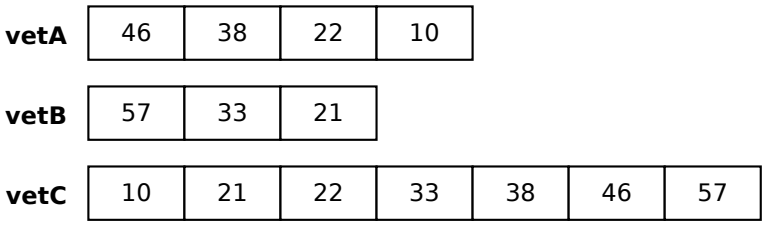
```
class Lista {
    CelulaLista inicio;
    CelulaLista fim;
}
class CelulaLista {
    CelulaPilha topo;
    CelulaLista prox;
}
class CelulaPilha {
    int elemento;
    CelulaPilha prox;
}
```



Resolução:

Questão 5 *(intercalação de vetores, Java)*

Implemente o método **public static int[] vetorOrdenado(int[] vetA, int[] vetB)** em Java, que recebe dois arrays de inteiros (tamanhos m e n) e retorna um terceiro array (tamanho m+n). O primeiro array (vetA) possui elementos pares, não primos, ordenados de forma decrescente. O segundo array (vetB) possui elementos ímpares, não primos, ordenados de forma decrescente. O array retornado deve possuir os valores dos dois arrays ordenados de forma CRESCENTE. A complexidade do método deve ser O(m+n). A figura abaixo apresenta um exemplo.



Resolução:

Questão 6 *(matriz flexível, C)*

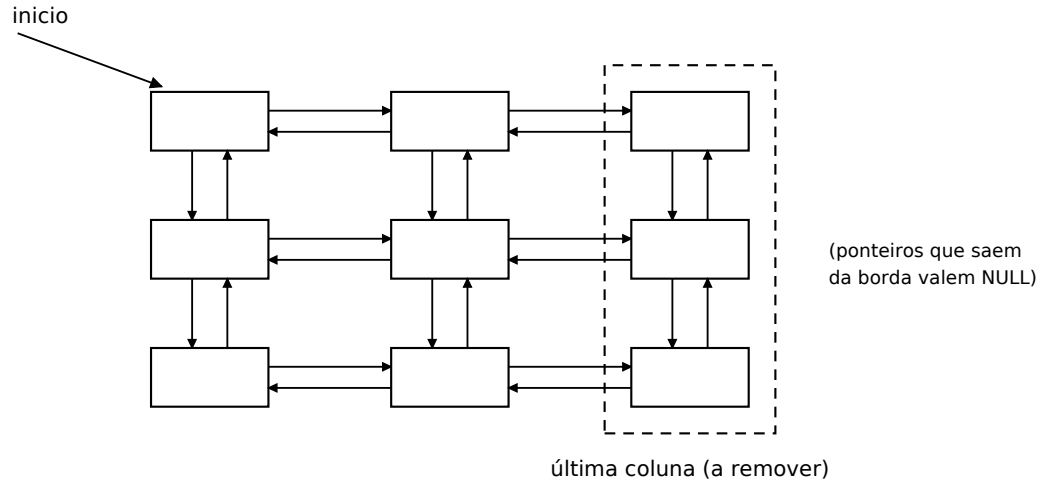
Considere uma estrutura de dados que representa uma matriz flexível, onde cada célula possui ponteiros para suas células vizinhas (superior, inferior, esquerda e direita). Essa estrutura permite o ajuste dinâmico das dimensões da matriz.

Implemente uma função em C que remova a **última coluna** dessa matriz flexível, ajustando os ponteiros dos elementos para manter a integridade da estrutura. Abaixo segue o código básico das estruturas utilizadas:

```
typedef struct Celula {
    int elemento;
    struct Celula *inf, *sup, *esq, *dir;
} Celula;

typedef struct {
    int linhas, colunas;
    Celula *inicio;
} Matriz;

void removerUltimaColuna(Matriz *matriz) {
    // IMPLEMENTAR
}
```



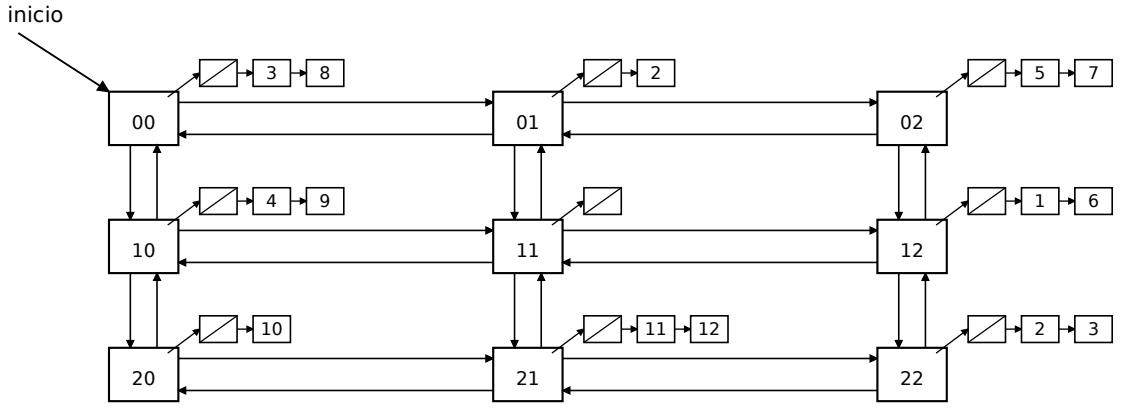
Resolução:

Questão 7 *(matriz de listas, Java)*

Seja a matriz de listas em Java apresentada abaixo, faça um método que **remove todos os números ímpares** existentes nas listas. Faça a análise de complexidade do seu método.

```
class Matriz {
    CelulaMat inicio;
}
class CelulaMat {
    CelulaMat prox, ant, sup, inf;
    Celula primeiro, ultimo;
    public CelulaMat() {
        prox = ant = sup = inf = null;
        primeiro = ultimo = new Celula();
    }
}
class Celula {
    int numero;
    Celula prox;
    public Celula() { this(0); }
    public Celula(int elemento) { this.numero = elemento; prox = null; }
}
```

Exemplo ilustrativo: cada CelulaMat aponta para uma lista com célula cabeça (riscada).



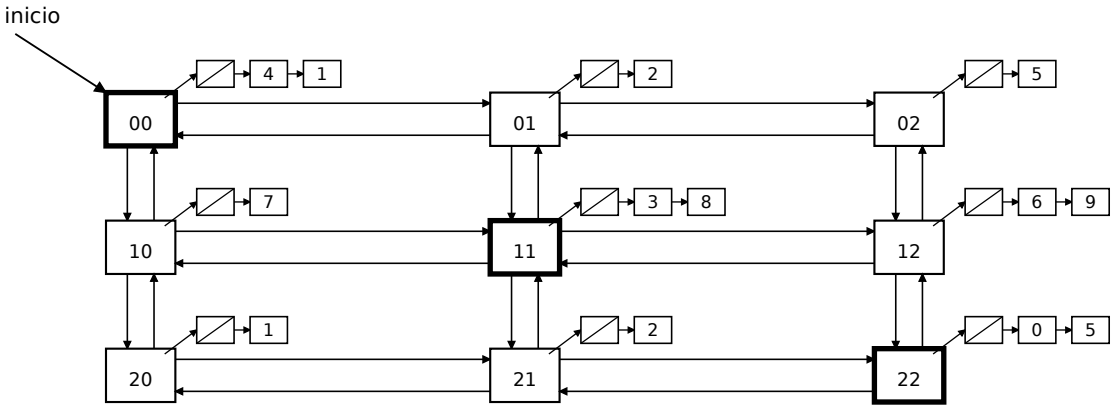
Resolução:

Questão 8 *(matriz de listas e lista dupla, Java)*

Considere a estrutura de classes abaixo, em Java, que representa uma matriz encadeada, onde cada célula da matriz (CelulaMatriz) contém uma lista de células (Celula). A classe Matriz possui um método **diagUnificada** que precisa ser implementado. Este método deve retornar um ponteiro para uma célula dupla (CelulaDupla) que será o início de uma lista duplamente encadeada contendo todas as células da diagonal principal da matriz (uma única lista duplamente encadeada, resultado da concatenação de todas as listas da diagonal principal). Considere que as listas da Matriz possuem Célula Cabeça, que devem ser desconsideradas na lista resultante.

```
class Matriz {
    CelulaMatriz inicio;
    int linha, coluna;
    CelulaDupla diagUnificada() {
        // implemente seu metodo aqui
    }
}
class CelulaMatriz {
    CelulaMatriz esq, dir, inf, sup;
    Celula inicio, fim;
}
class Celula {
    int elemento;
    Celula prox;
}
class CelulaDupla {
    int elemento;
    CelulaDupla prox, ant;
}
```

Exemplo ilustrativo: as células da diagonal principal estão com borda grossa.



Resolução:

Questão 9 *(árvore binária e matriz flexível, C)*

Dado o código abaixo, faça a função **Celula* encontrarRepetidos(No* raiz, CelulaMatriz* inicio)** em C, que recebe o endereço do nó raiz de uma árvore binária e o endereço da primeira célula de uma matriz flexível e retorna o endereço do nó cabeça de uma lista simples ordenada de forma **decrescente** contendo todos os números que existem tanto na árvore como na matriz flexível. A célula da lista, o nó da árvore e a célula da matriz possuem apenas um elemento inteiro e os devidos ponteiros dessas estruturas, conforme visto em sala de aula. Sabe-se que a árvore não tem elementos repetidos; sabe-se a mesma coisa para a matriz. Faça a análise de complexidade da sua função.

```
typedef struct Celula {
    int numero;
    struct Celula *prox;
} Celula;

typedef struct CelulaMatriz {
    int numero;
    struct CelulaMatriz *prox, *ant;
    struct CelulaMatriz *sup, *inf;
} CelulaMatriz;

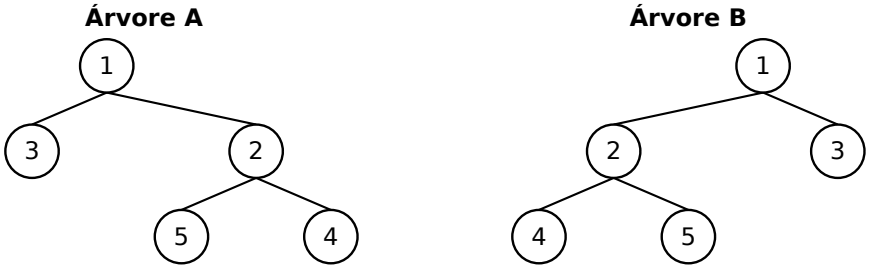
typedef struct No {
    int numero;
    struct No *esq, *dir;
} No;
```

Resolução:

Questão 10 *(árvore binária, Java)*

Considere duas árvores binárias A e B que armazenam inteiros e implemente o método recursivo **private boolean isEspelho(No a, No b)** dentro da classe Arvore. Duas árvores são consideradas Espelhadas (ou simetricamente equivalentes) se, e somente se, o valor na raiz de A for igual ao valor na raiz de B, a subárvore esquerda de A for espelhada à subárvore direita de B, e a subárvore direita de A for espelhada à subárvore esquerda de B (o que inclui o caso base de ambas serem vazias). Use a figura como referência visual. Apresente, em função de Θ, a complexidade do seu método.

```
class No {
    public int elemento;
    public No esq, dir;
}
class Arvore {
    private No raiz;
    public boolean isEspelho(Arvore outra) {
        return isEspelho(this.raiz, outra.raiz);
    }
    private boolean isEspelho(No a, No b) {
        // IMPLEMENTAR
    }
}
```



Resolução:

Questão 11 *(árvore binária de pesquisa, Java)*

Considere a implementação de uma árvore binária de busca (BST) em que os nós armazenam números inteiros. Na BST vista em sala de aula, não eram aceitos elementos repetidos. Para este exercício, você deve modificar a BST para que, ao inserir um número já existente, em vez de ignorar a inserção, a árvore armazene a quantidade de vezes que cada número foi inserido. Você deve implementar o método **void inserir(int x)**. Você pode modificar os atributos da árvore. Apresente a complexidade do código em função de Θ.

```
class No {
    public int elemento;
    public No esq, dir;
}

public class Arvore {
    private No raiz;
    public void inserir(int x) {
        // Implemente o metodo aqui
    }
}
```

Resolução:

Questão 12 *(ordenação com árvore, Java)*

Um algoritmo de ordenação é o **TreeSort**, que insere os elementos do array em uma árvore binária e utiliza um "caminhar" para ordenar os elementos do array. Implemente o TreeSort e faça a análise de complexidade do mesmo.

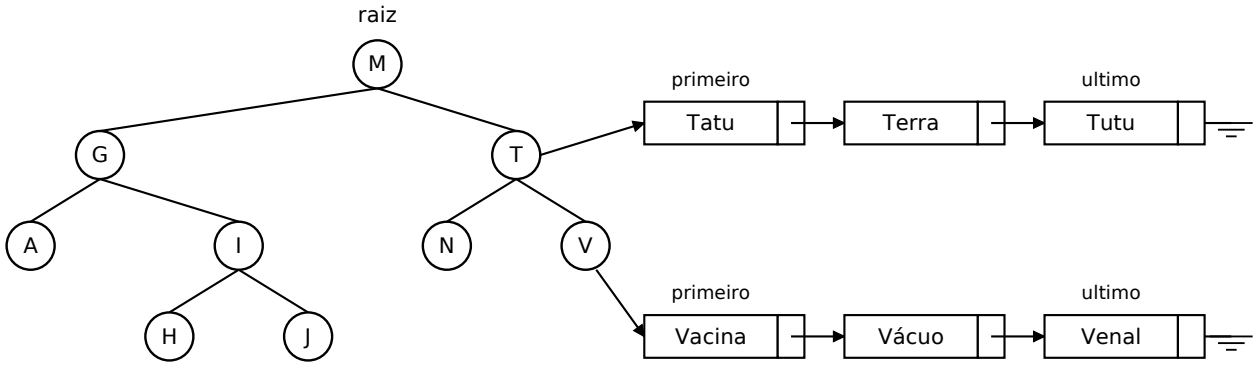
Resolução:

Questão 13 *(árvore de listas, Java)*

Considere uma estrutura de dados que organiza palavras de um dicionário usando uma árvore de listas. Nessa estrutura, cada nó representa a primeira letra de um conjunto de palavras, enquanto as palavras são armazenadas em listas encadeadas, mantidas em ordem alfabética. Caso o nó para a letra inicial de uma nova palavra não exista, ele deve ser criado e adicionado à árvore, garantindo a correta ordenação das palavras dentro da lista de cada nó.

Implemente, em Java, o método **inserir** da classe Dicionario, responsável por adicionar uma nova palavra a essa estrutura de dados.

```
class Celula {
    public String palavra;
    public Celula prox;
}
class No {
    public char letra;
    public No esq, dir;
    public Celula primeiro, ultimo;
}
class Dicionario {
    private No raiz;
    public void inserir(String palavra) {
        // IMPLEMENTAR
    }
}
```



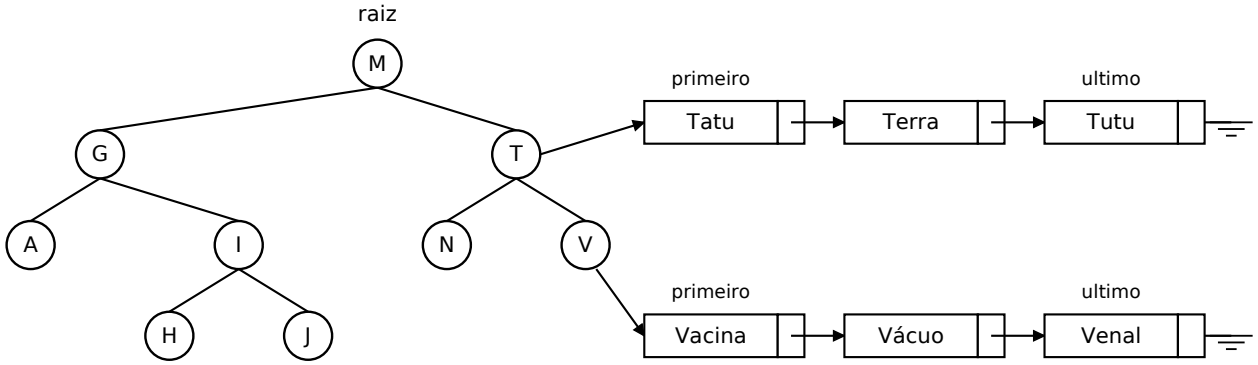
(as outras letras também têm suas listas; foram omitidas)

Resolução:

Questão 14 *(árvore de listas, Java)*

Considere uma estrutura de dados do tipo árvore de listas, conforme a figura, usada para armazenar palavras de um dicionário. Os nós da árvore guardam a primeira letra da palavra, enquanto as palavras são armazenadas nas listas correspondentes em ordem alfabética. Implemente, em Java, o método para **remover uma palavra** na estrutura. O método deve pressupor que todos os nós com as letras iniciais existem na árvore. Na sua solução, considere o código apresentado e que a lista **não possui célula cabeça**. Explique como a ordenação da lista tem impacto na performance da sua solução. Apresente, em função de Θ, a complexidade do seu método.

```
class Celula {
    public String palavra;
    public Celula prox;
}
class No {
    public char letra;
    public No esq, dir;
    public Celula primeiro, ultimo;
}
class Dicionario {
    private No raiz;
    public void remover(String palavra) {
        // IMPLEMENTAR
    }
}
```



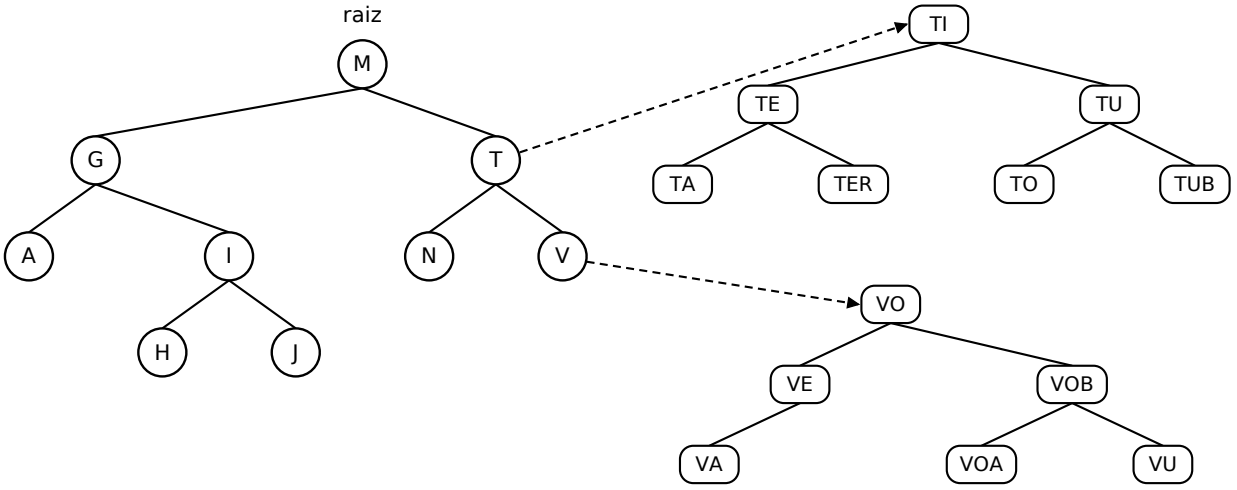
(as outras letras também têm suas listas; foram omitidas)

Resolução:

Questão 15 *(árvore de árvore, Java)*

Na árvore de árvore (ilustrada e codificada abaixo), implemente o método **int contarPalavras(char primeiro, char ultimo)** em Java, que recebe dois caracteres e retorna o número de palavras que começam com o caractere *primeiro* e terminam com o caractere *ultimo*. Faça a análise de complexidade do seu método.

```
class ArvoreArvore {
    No raiz;
}
class No {
    char letra;
    No esq, dir;
    No2 raiz;
}
class No2 {
    String palavra;
    No2 esq, dir;
}
```



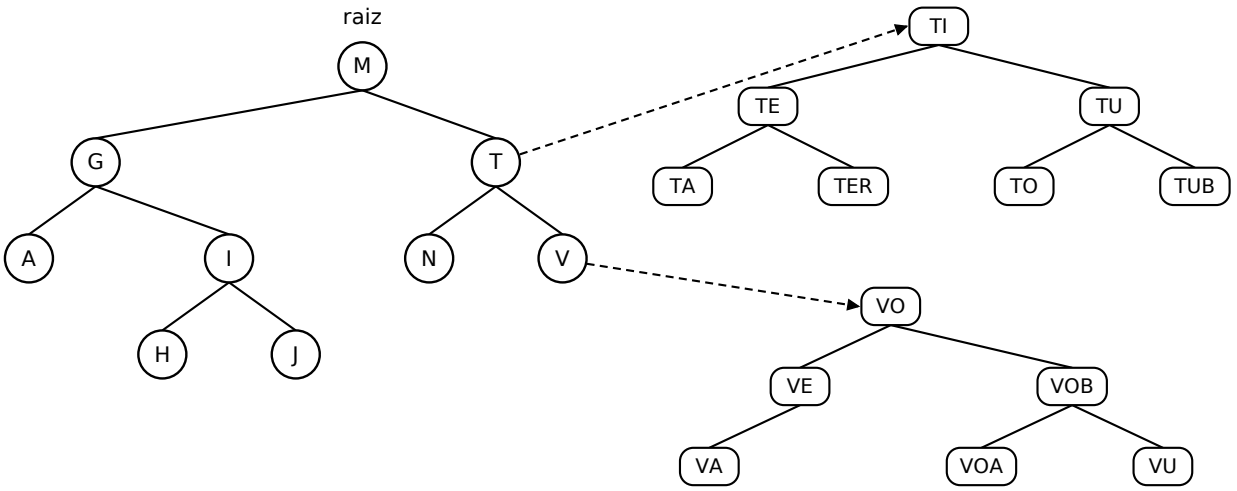
(cada No aponta, pelo atributo raiz, para uma árvore de No2; aqui só T e V foram desenhados)

Resolução:

Questão 16 *(árvore de árvore, Java)*

Na árvore de árvore (ilustrada e codificada abaixo), implemente o método **int contarPalavras(String padrao)** em Java, que recebe uma String e retorna o número de palavras que começam com a mesma letra e possuem a mesma quantidade de caracteres. Ex.: se a palavra passada por parâmetro fosse "PROVA", deveriam ser retornadas quantas palavras iniciam com P e possuem 5 caracteres. Obs.: somente caracteres maiúsculos foram utilizados. Faça a análise de complexidade do seu método.

```
class ArvoreArvore {
    No raiz;
}
class No {
    char letra;
    No esq, dir;
    No2 raiz;
}
class No2 {
    String palavra;
    No2 esq, dir;
}
```



(cada No aponta, pelo atributo raiz, para uma árvore de No2; aqui só T e V foram desenhados)

Resolução:

Questão 17 *(ordenação e árvores, verdadeiro ou falso)*

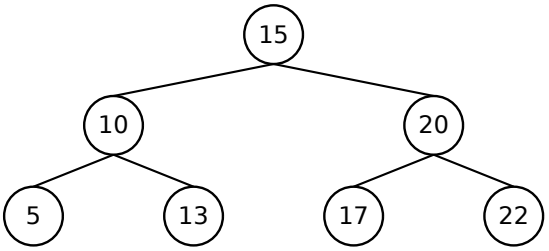
Analise as afirmações a seguir e diga se cada uma é verdadeira ou falsa, com uma justificativa detalhada. Somente as respostas com a justificativa completa serão consideradas.

(a) Você está organizando o catálogo de uma biblioteca e sabe que os livros estão ordenados em ordem crescente por título, com exceção de um único livro fora de posição. Para reorganizar o catálogo completamente em ordem crescente, considerando os algoritmos *merge sort, insertion sort, radix sort, heapsort* e *counting sort*, o algoritmo mais adequado é o *merge sort*.

(b) Você está trabalhando em um dispositivo com pouca memória RAM e tem urgência em ordenar um conjunto de dados de 50 MB. Após carregar os dados em memória, o dispositivo ficou com apenas 5 MB livres. Considerando os algoritmos *merge sort, insertion sort, radix sort, heapsort* e *counting sort*, o mais adequado para essa tarefa é o *heapsort*.

(c) O *quicksort* é um dos algoritmos de ordenação mais usados por causa da sua eficiência em muitos casos. Sua complexidade de tempo é $\Theta(n \log n)$ em todos os casos (melhor, médio e pior).

(d) A árvore binária de pesquisa abaixo possui a seguinte sequência no caminhamento pós-ordem: 15, 10, 5, 13, 20, 17, 22.



(e) Todo *heap* com mais de zero elementos também pode ser considerado uma árvore binária de pesquisa.

Gabarito

Existem outras respostas corretas. O que importa é a ideia e a complexidade bater com o código.

Questão 1: inserirFim (lista dupla circular)

Como a lista é circular, a última célula é sempre **primeiro->ant**. Não é preciso andar pela lista.

```
void inserirFim(int x) {
    CelulaDupla* nova = novaCelulaDupla(x);
    CelulaDupla* ultima = primeiro->ant;  // lista vazia: e a propria cabeca

    nova->ant  = ultima;
    nova->prox = primeiro;     // fecha o circulo
    ultima->prox  = nova;
    primeiro->ant = nova;
}
```

Com a lista vazia, a cabeça aponta para ela mesma (prox e ant), então o mesmo código funciona.

Complexidade: $\Theta(1)$, pois o número de operações não depende do tamanho da lista.

Questão 2: meiose (lista simples)

```
void meiose() {
    Celula* i = inicio;
    while (i != NULL) {
        int metade = i->elemento / 2;     // divisao inteira: 5/2 = 2, 1/2 = 0
        Celula* nova = (Celula*) malloc(sizeof(Celula));
        nova->elemento = metade;
        nova->prox = i->prox;
        i->elemento = metade;
        i->prox = nova;
        if (i == fim) {
            fim = nova;
        }
        i = nova->prox;                   // pula a celula que acabou de ser criada
    }
}
```

Pela figura, **inicio** aponta direto para a primeira célula com dado (sem célula cabeça). Se a lista da sua turma usar célula cabeça, basta começar com **i = inicio->prox**.

Complexidade: $\Theta(n)$, pois cada uma das n células é visitada uma vez e cada passo é $\Theta(1)$.

Questão 3: intercalarReverso (listas duplas)

Primeiro andamos até a última célula de cada lista. Depois voltamos pelos ponteiros **ant**, pegando um elemento de cada lista de forma alternada. Quando uma lista acaba, o resto vem da outra.

```
CelulaDupla intercalarReverso(CelulaDupla c1, CelulaDupla c2) {
    CelulaDupla i = c1;
    while (i.prox != null) i = i.prox;     // ultima celula de c1
    CelulaDupla j = c2;
    while (j.prox != null) j = j.prox;     // ultima celula de c2

    CelulaDupla c3 = new CelulaDupla();    // celula cabeca da nova lista
    CelulaDupla fim = c3;
    boolean vezDaPrimeira = true;

    while (i != c1 || j != c2) {           // enquanto alguma lista tiver elementos
        CelulaDupla origem;
        if ((vezDaPrimeira && i != c1) || j == c2) {
            origem = i;
            i = i.ant;
        } else {
            origem = j;
            j = j.ant;
        }
        CelulaDupla nova = new CelulaDupla();
        nova.elemento = origem.elemento;
        nova.ant = fim;
        fim.prox = nova;
        fim = nova;
        vezDaPrimeira = !vezDaPrimeira;
    }
    return c3;
}
```

Teste com o exemplo: c1 = 3, 7, 5 e c2 = 1, 9 geram 5, 9, 7, 1, 3.

Complexidade: $\Theta(n + m)$, sendo n e m os tamanhos das listas. Achar os finais custa $\Theta(n + m)$ e montar a nova lista custa mais $\Theta(n + m)$.

Questão 4: maiorPilha (lista de pilhas)

```
CelulaLista maiorPilha() {
    CelulaLista maior = null;
    int maiorTam = -1;
    for (CelulaLista i = inicio; i != null; i = i.prox) {
        int tam = 0;
        for (CelulaPilha p = i.topo; p != null; p = p.prox) {
            tam++;
        }
        if (tam > maiorTam) {   // ">" (e nao ">="): no empate fica a primeira
            maiorTam = tam;
            maior = i;
        }
    }
    return maior;
}
```

Pela figura, **inicio** já aponta para uma célula com pilha (sem célula cabeça). Se houvesse cabeça, o laço começaria em **inicio.prox**. No desenho, a resposta seria a

primeira célula (pilha com 3 itens).

Complexidade: $\Theta(k + N)$, sendo k o número de células da lista e N o total de elementos somando todas as pilhas. Cada elemento de pilha é contado uma vez.

Questão 5: vetorOrdenado (intercalação)

Como os dois vetores estão em ordem decrescente, os menores valores ficam no fim. Então lemos os dois vetores de trás para frente e fazemos a intercalação, como no merge.

```
public static int[] vetorOrdenado(int[] vetA, int[] vetB) {
    int m = vetA.length, n = vetB.length;
    int[] vetC = new int[m + n];
    int i = m - 1, j = n - 1, k = 0;

    while (i >= 0 && j >= 0) {
        if (vetA[i] < vetB[j]) {
            vetC[k++] = vetA[i--];
        } else {
            vetC[k++] = vetB[j--];
        }
    }
    while (i >= 0) vetC[k++] = vetA[i--];
    while (j >= 0) vetC[k++] = vetB[j--];
    return vetC;
}
```

Como um vetor só tem pares e o outro só ímpares, nunca há valores iguais.

Complexidade: $\Theta(m + n)$, pois cada elemento é copiado uma única vez.

Questão 6: removerUltimaColuna (matriz flexível)

Andamos pela primeira linha até a célula mais à direita (topo da última coluna). Depois descemos pela coluna: para cada célula, a vizinha da esquerda passa a ter **dir = NULL** e a célula é liberada.

```
void removerUltimaColuna(Matriz *matriz) {
    if (matriz == NULL || matriz->inicio == NULL) return;

    Celula *i = matriz->inicio;
    while (i->dir != NULL) {          // vai ate o topo da ultima coluna
        i = i->dir;
    }

    while (i != NULL) {               // desce pela ultima coluna
        Celula *abaixo = i->inf;
        if (i->esq != NULL) {
            i->esq->dir = NULL;       // a coluna da esquerda vira a ultima
        }
        free(i);
        i = abaixo;
    }

    matriz->colunas--;
    if (matriz->colunas == 0) {       // a matriz tinha uma coluna so
        matriz->inicio = NULL;
        matriz->linhas = 0;
```

```
    }
}
```

Complexidade: Θ(c + l), sendo c o número de colunas (para chegar à última) e l o número de linhas (para remover cada célula da coluna).

Questão 7: remover ímpares (matriz de listas)

Percorremos a matriz linha por linha (inf) e coluna por coluna (prox). Em cada lista, como existe célula cabeça, olhamos sempre a célula seguinte (**i.prox**), o que facilita remover.

```
public void removerImpares() {
    for (CelulaMat linha = inicio; linha != null; linha = linha.inf) {
        for (CelulaMat c = linha; c != null; c = c.prox) {
            Celula i = c.primeiro;              // celula cabeca
            while (i.prox != null) {
                if (i.prox.numero % 2 != 0) {   // impar (vale para negativos tambem)
                    Celula tmp = i.prox;
                    i.prox = tmp.prox;
                    tmp.prox = null;
                } else {
                    i = i.prox;
                }
            }
            c.ultimo = i;   // i e a ultima celula que sobrou (ou a cabeca)
        }
    }
}
```

Complexidade: Θ(l·c + N), sendo l·c o número de células da matriz e N o total de números guardados em todas as listas. Cada número é visitado uma vez.

Questão 8: diagUnificada (matriz de listas)

Começamos em **inicio** (posição 0,0). Para ir à próxima posição da diagonal, andamos uma vez para a direita e uma vez para baixo. Em cada lista, pulamos a célula cabeça (**inicio.prox**) e copiamos os elementos para novas células duplas.

```
CelulaDupla diagUnificada() {
    CelulaDupla primeiro = null, ultimo = null;
    CelulaMatriz d = inicio;

    while (d != null) {
        for (Celula i = d.inicio.prox; i != null; i = i.prox) {  // pula a cabeca
            CelulaDupla nova = new CelulaDupla();
            nova.elemento = i.elemento;
            if (primeiro == null) {
                primeiro = nova;
            } else {
                ultimo.prox = nova;
                nova.ant = ultimo;
            }
            ultimo = nova;
        }
        d = (d.dir != null) ? d.dir.inf : null;   // proxima posicao da diagonal
    }
    return primeiro;   // null se a diagonal nao tiver elementos
}
```

Como o enunciado pede para desconsiderar as cabeças, o retorno é a primeira célula com dado. Se o professor preferir que a nova lista também tenha cabeça, basta criar uma CelulaDupla vazia no começo e retornar ela.

Complexidade: Θ(min(l, c) + T), sendo min(l, c) o tamanho da diagonal e T o total de elementos nas listas da diagonal.

Questão 9: encontrarRepetidos (árvore e matriz)

Para cada célula da matriz, procuramos o número na árvore. Se achar, inserimos na lista em ordem decrescente. Como a matriz não tem repetidos, a lista também não terá. Aqui a árvore foi tratada como árvore binária de pesquisa (como vista em sala).

```
Celula* novaCelula(int x) {
    Celula* nova = (Celula*) malloc(sizeof(Celula));
    nova->numero = x;
    nova->prox = NULL;
    return nova;
}

int existeNaArvore(No* i, int x) {
    while (i != NULL) {
        if (x == i->numero) return 1;
        i = (x < i->numero) ? i->esq : i->dir;
    }
    return 0;
}

void inserirDecrescente(Celula* cabeca, int x) {
    Celula* i = cabeca;
    while (i->prox != NULL && i->prox->numero > x) {
        i = i->prox;
    }
    Celula* nova = novaCelula(x);
    nova->prox = i->prox;
    i->prox = nova;
}

Celula* encontrarRepetidos(No* raiz, CelulaMatriz* inicio) {
    Celula* cabeca = novaCelula(0);                 // celula cabeca da lista
    for (CelulaMatriz* lin = inicio; lin != NULL; lin = lin->inf) {
        for (CelulaMatriz* j = lin; j != NULL; j = j->prox) {
            if (existeNaArvore(raiz, j->numero)) {
                inserirDecrescente(cabeca, j->numero);
            }
        }
    }
    return cabeca;
}
```

Complexidade: seja k o número de células da matriz, n o número de nós da árvore (altura h) e r o tamanho final da lista. O custo é O(k·(h + r)). Com árvore balanceada, h = log n, ficando O(k·(log n + r)). No pior caso (árvore degenerada), h = n e o custo fica O(k·n). Se a árvore não for de pesquisa, a busca precisa olhar os dois lados e custa Θ(n) para cada célula, ou seja, Θ(k·n) no total.

Questão 10: isEspelho (árvores binárias)

```
private boolean isEspelho(No a, No b) {
    if (a == null && b == null) return true;    // as duas vazias
    if (a == null || b == null) return false;   // so uma vazia
    if (a.elemento != b.elemento) return false;
    return isEspelho(a.esq, b.dir) && isEspelho(a.dir, b.esq);
}
```

Na figura, A e B são espelhadas: 1 = 1; a esquerda de A (3) bate com a direita de B (3); a direita de A (2 com 5 e 4) bate com a esquerda de B (2 com 4 e 5) espelhada.

Complexidade: cada par de nós é comparado no máximo uma vez. Pior caso Θ(n), quando as árvores são espelhadas e todos os n nós são visitados. Melhor caso Θ(1), quando as raízes já são diferentes.

Questão 11: inserir com contador (BST)

Adicionamos o atributo **quantidade** no No. Se o número já existir, só aumentamos esse contador.

```
class No {
    public int elemento;
    public int quantidade;      // novo atributo
    public No esq, dir;
    public No(int x) {
        elemento = x;
        quantidade = 1;
        esq = dir = null;
    }
}

public class Arvore {
    private No raiz;

    public void inserir(int x) {
        raiz = inserir(x, raiz);
    }

    private No inserir(int x, No i) {
        if (i == null) {
            i = new No(x);
        } else if (x < i.elemento) {
            i.esq = inserir(x, i.esq);
        } else if (x > i.elemento) {
            i.dir = inserir(x, i.dir);
        } else {
            i.quantidade++;     // ja existe: conta mais uma vez
        }
        return i;
    }
}
```

Complexidade: Θ(h), sendo h a altura. Melhor caso Θ(1) (o número está na raiz), caso médio ou árvore balanceada Θ(log n) e pior caso Θ(n) (árvore degenerada, tipo lista).

Questão 12: TreeSort

Inserimos tudo numa árvore binária de pesquisa e depois fazemos o caminhamento central (em ordem), que visita os valores do menor para o maior, escrevendo de volta no array.

```
class TreeSort {
    private class No {
        int elemento;
        No esq, dir;
        No(int x) { elemento = x; }
    }
    private No raiz;
    private int pos;

    public void ordenar(int[] array) {
        raiz = null;
        for (int x : array) {
            raiz = inserir(raiz, x);
        }
        pos = 0;
        caminharCentral(raiz, array);
    }

    private No inserir(No i, int x) {
        if (i == null) return new No(x);
        if (x < i.elemento) i.esq = inserir(i.esq, x);
        else i.dir = inserir(i.dir, x);   // repetidos vao para a direita
        return i;
    }

    private void caminharCentral(No i, int[] array) {
        if (i != null) {
            caminharCentral(i.esq, array);
            array[pos++] = i.elemento;
            caminharCentral(i.dir, array);
        }
    }
}
```

Complexidade: o caminhamento custa $\Theta(n)$. As inserções custam $\Theta(n \log n)$ no melhor caso e no caso médio (árvore mais ou menos balanceada). No pior caso, quando o array já está ordenado (crescente ou decrescente), a árvore vira uma lista e as inserções custam $1 + 2 + \ldots + n$, ou seja, $\Theta(n^2)$. Usando uma árvore balanceada (AVL, por exemplo), o pior caso cai para $\Theta(n \log n)$. Memória extra: $\Theta(n)$.

Questão 13: inserir (árvore de listas)

Procuramos a letra na árvore guardando o pai. Se não existir, criamos o nó no lugar certo. Depois inserimos a palavra na lista (sem célula cabeça) mantendo a ordem alfabética.

```
public void inserir(String palavra) {
    char letra = palavra.charAt(0);
    No no = raiz, pai = null;
    while (no != null && no.letra != letra) {
        pai = no;
        no = (letra < no.letra) ? no.esq : no.dir;
    }
```

```
    if (no == null) {                       // letra ainda nao existe
        no = new No();
        no.letra = letra;
        if (pai == null) raiz = no;
        else if (letra < pai.letra) pai.esq = no;
        else pai.dir = no;
    }

    Celula nova = new Celula();
    nova.palavra = palavra;
    Celula ant = null, i = no.primeiro;
    while (i != null && i.palavra.compareTo(palavra) < 0) {
        ant = i;
        i = i.prox;
    }
    nova.prox = i;
    if (ant == null) no.primeiro = nova;    // entra no comeco
    else ant.prox = nova;
    if (i == null) no.ultimo = nova;        // entra no fim
}
```

Complexidade: Θ(h + m), sendo h a altura da árvore de letras e m o tamanho da lista da letra. Como existem no máximo 26 letras, h é limitado por uma constante, então o custo fica O(m). Melhor caso Θ(1) (palavra entra no começo de uma lista perto da raiz) e pior caso Θ(m) (entra no fim da lista).

Questão 14: remover (árvore de listas)

```
public void remover(String palavra) {
    char letra = palavra.charAt(0);
    No no = raiz;
    while (no.letra != letra) {             // o enunciado garante que a letra existe
        no = (letra < no.letra) ? no.esq : no.dir;
    }

    Celula ant = null, i = no.primeiro;
    while (i != null && i.palavra.compareTo(palavra) < 0) {
        ant = i;
        i = i.prox;
    }
    if (i == null || !i.palavra.equals(palavra)) {
        return;                             // palavra nao esta na lista
    }

    if (ant == null) no.primeiro = i.prox;  // removendo a primeira
    else ant.prox = i.prox;
    if (i == no.ultimo) no.ultimo = ant;    // removendo a ultima
    i.prox = null;
}
```

Impacto da ordenação: como a lista está em ordem alfabética, a busca pode parar assim que encontrar uma palavra "maior" que a procurada. Assim, quando a palavra não existe, não é preciso percorrer a lista inteira. Mesmo assim, no pior caso (palavra no fim ou maior que todas) ainda é preciso andar a lista toda, porque lista encadeada não permite pesquisa binária.

Complexidade: achar a letra custa no máximo 26 passos, ou seja, Θ(1). Na lista de tamanho m: melhor caso Θ(1) (primeira palavra) e pior caso Θ(m).

Questão 15: contarPalavras(char primeiro, char ultimo)

Achamos a letra *primeiro* na árvore de letras. Depois percorremos toda a árvore de palavras (No2) daquela letra contando as que terminam com *ultimo*. Não dá para cortar caminho, porque a ordem da árvore de No2 é pela palavra toda, e não pela última letra.

```
public int contarPalavras(char primeiro, char ultimo) {
    No no = raiz;
    while (no != null && no.letra != primeiro) {
        no = (primeiro < no.letra) ? no.esq : no.dir;
    }
    if (no == null) return 0;
    return contar(no.raiz, ultimo);
}

private int contar(No2 i, char ultimo) {
    if (i == null) return 0;
    int resp = 0;
    if (i.palavra.charAt(i.palavra.length() - 1) == ultimo) {
        resp = 1;
    }
    return resp + contar(i.esq, ultimo) + contar(i.dir, ultimo);
}
```

Complexidade: achar a letra custa no máximo 26 passos, Θ(1). Percorrer a árvore de palavras da letra custa Θ(p), sendo p o número de palavras que começam com aquela letra. Total: Θ(p).

Questão 16: contarPalavras(String padrao)

```
public int contarPalavras(String padrao) {
    char letra = padrao.charAt(0);
    int tamanho = padrao.length();
    No no = raiz;
    while (no != null && no.letra != letra) {
        no = (letra < no.letra) ? no.esq : no.dir;
    }
    if (no == null) return 0;
    return contar(no.raiz, tamanho);
}

private int contar(No2 i, int tamanho) {
    if (i == null) return 0;
    int resp = (i.palavra.length() == tamanho) ? 1 : 0;
    return resp + contar(i.esq, tamanho) + contar(i.dir, tamanho);
}
```

Complexidade: igual à questão anterior. Achar a letra é Θ(1) (no máximo 26 nós) e percorrer a árvore de palavras da letra é Θ(p). Total: Θ(p). Também não dá para cortar caminho, porque a árvore de No2 é ordenada pela palavra, não pelo tamanho.

Questão 17: verdadeiro ou falso

(a) Falsa. Com só um livro fora do lugar, o melhor é o *insertion sort*: em dados quase ordenados ele faz pouquíssimas trocas e roda em tempo próximo de $\Theta(n)$. O *merge sort* sempre gasta $\Theta(n \log n)$, mesmo com os dados quase prontos, e ainda usa memória extra. *Radix* e *counting sort* não fazem sentido para títulos (textos), e o *heapsort* também é $\Theta(n \log n)$.

(b) Verdadeira. O *heapsort* ordena no próprio vetor (memória extra $\Theta(1)$) e garante $\Theta(n \log n)$ até no pior caso. O *merge sort* precisaria de outro vetor do mesmo tamanho (mais 50 MB). *Counting* e *radix sort* também usam vetores auxiliares grandes. O *insertion sort* usa pouca memória, mas é $\Theta(n^2)$, ruim para quem tem urgência.

(c) Falsa. O *quicksort* é $\Theta(n \log n)$ no melhor caso e no caso médio, mas no pior caso é $\Theta(n^2)$. Isso acontece quando o pivô escolhido é sempre o menor ou o maior elemento (por exemplo, pivô no início com o vetor já ordenado), deixando as partições muito desbalanceadas.

(d) Falsa. A sequência 15, 10, 5, 13, 20, 17, 22 é o caminhamento **pré-ordem** (raiz, esquerda, direita). O pós-ordem (esquerda, direita, raiz) é: 5, 13, 10, 17, 22, 20, 15.

(e) Falsa. No heap (de máximo, por exemplo) o pai é maior que os **dois** filhos; na árvore de pesquisa, o filho da esquerda deve ser menor e o da direita maior que o pai. Contraexemplo: o heap com raiz 10 e filhos 5 (esquerda) e 3 (direita) é válido, mas não é árvore de pesquisa, porque 3 está à direita de 10 sendo menor. Algumas árvores pequenas podem ser as duas coisas, mas não **todo** heap.